ShopNow RAG - Short Guide

How the simple project works: diagrams, steps and the Python ideas behind them (LangChain + OpenAI + FAISS + Gradio)

our code | data / search | OpenAI call | decision / limit

1. The big picture

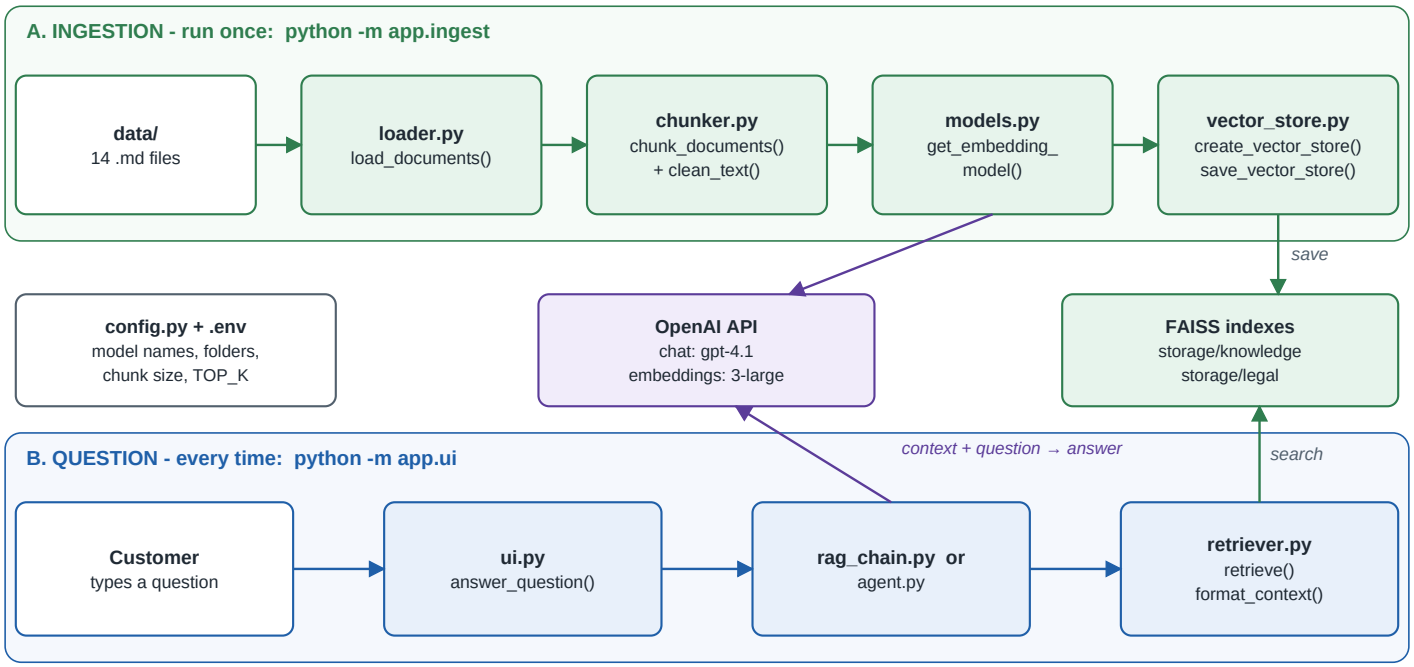


Figure 1 - A builds the search indexes once. B answers every question using them.

- **RAG** = find the right text first (*retrieval*), then let the LLM write the answer from it (*generation*).
- **Two models**: the *embedding model* turns text into numbers for searching; the *LLM* reads and writes text.
- **Two ways to answer**: the simple chain always searches, then answers. The agent decides itself which tools to call.

2. Files

Step	File	Function(s)	Job
0	config.py	-	Settings from .env: models, folders, CHUNK_SIZE, TOP_K
1	models.py	get_llm(), get_embedding_model()	The two OpenAI models
2	loader.py	load_documents()	Read .md files → Documents
3	preprocess.py	clean_text(), remove_card_numbers()	Tidy documents; protect card numbers
4	chunker.py	chunk_documents()	Split by heading, then by size
5-6	vector_store.py	create / save / load_vector_store()	Embed chunks, FAISS index on disk
-	ingest.py	build_all_indexes()	Runs steps 2 to 6
7	retriever.py	retrieve(), format_context()	Search; turn results into context text
8	prompts.py	create_qa_prompt(), create_agent_prompt()	Instructions for the LLM
9	rag_chain.py	create_qa_chain(), generate_answer()	Simple RAG: retrieve → generate
10	agent.py	create_tools(), run_agent()	Tools + the agent loop
11	ui.py	answer_question()	Gradio web page

3. Ingestion: documents → search index

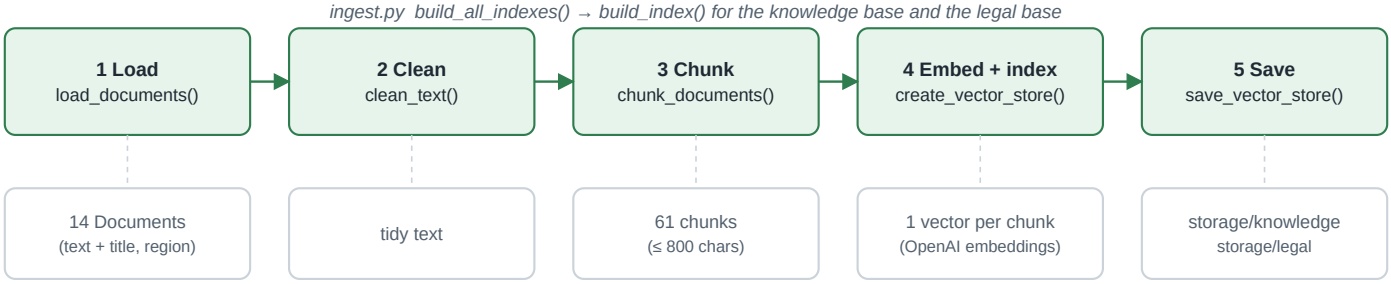


Figure 2 - Each box is one function. Under it: what it produces.

- **Load**: each file has a YAML header (title, region). It goes into the Document's metadata, so search can filter by region.
- **Chunk**: one heading = one topic; long parts are cut to 800 characters with 100 overlap. The title is added in front of each chunk.
- **Embed + index**: FAISS.from_documents() sends the chunks to OpenAI embeddings and stores the vectors.
- **Save**: the index is written to storage/, so the next start does not pay for embeddings again.

4. Answering with the simple RAG chain

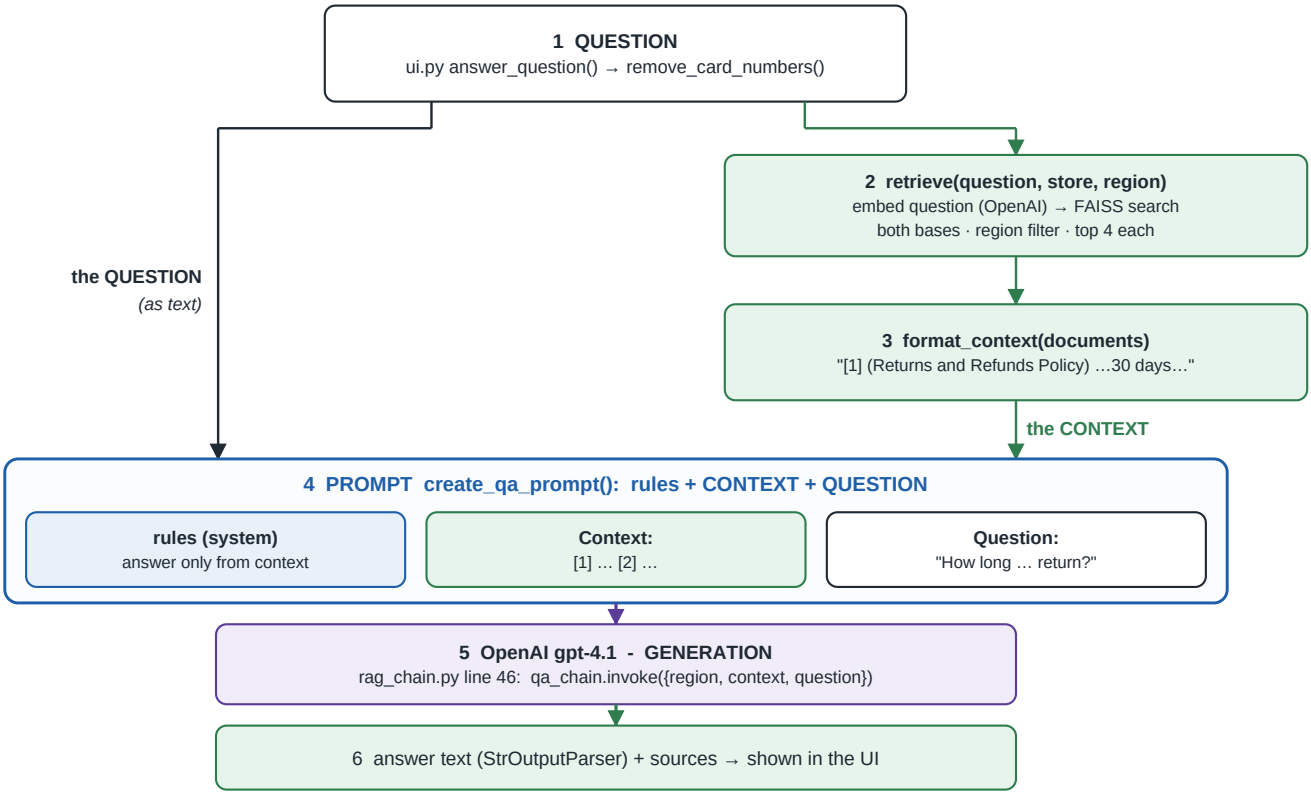


Figure 3 - The question is used twice: to search (right) and as text in the prompt (left). They meet in one OpenAI call.

```
documents = retrieve(question, stores["knowledge"], region) + retrieve(question, stores["legal"], region)
context = format_context(documents)                                  # chunks -> one text
answer = qa_chain.invoke({"region": region, "context": context, "question": question})   # OpenAI call
```

`qa_chain` is `prompt | llm | StrOutputParser()`: fill the prompt → call gpt-4.1 → keep only the answer text.

5. Answering with the agent (tool calling)

The LLM gets a list of **tools** (Python functions). It does not run them; it *asks* us to, and our loop runs them.

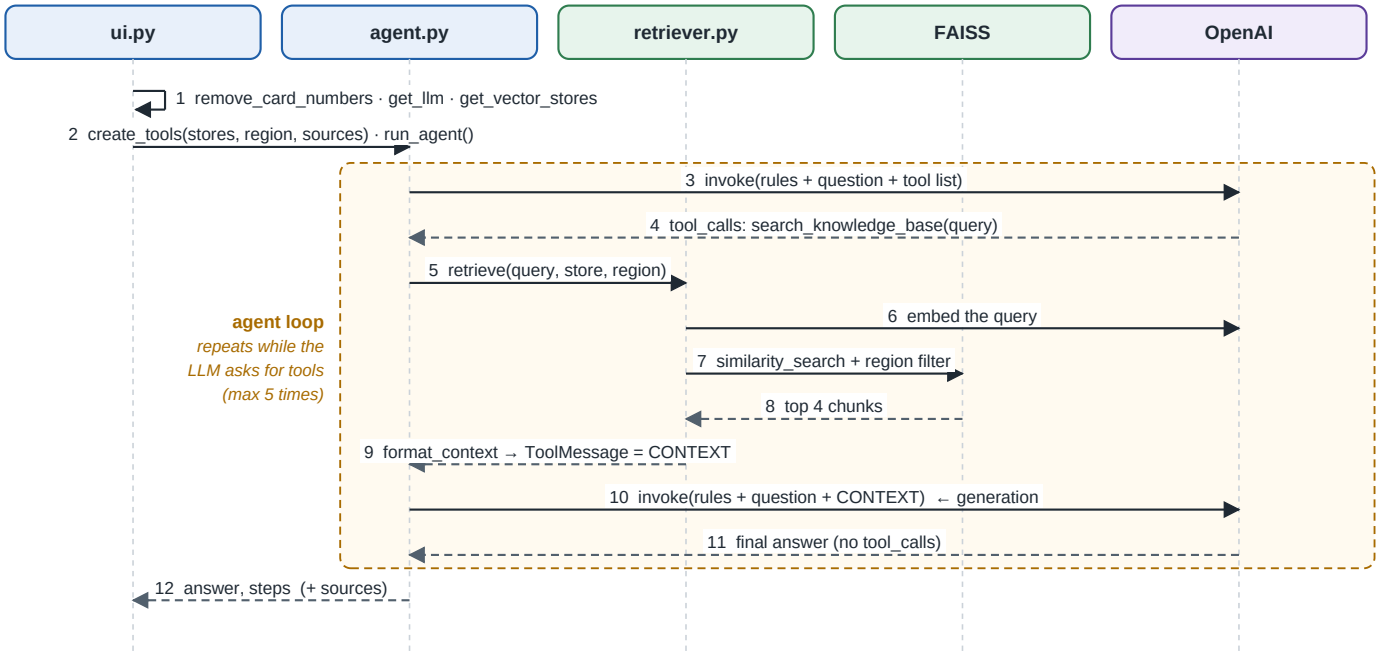


Figure 4 - One agent question, call by call. Dashed = returned value.

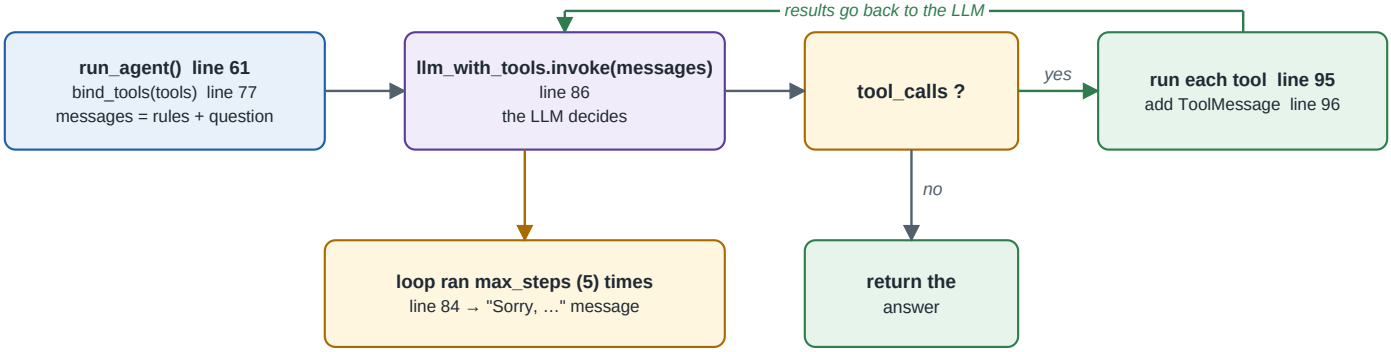


Figure 5 - run_agent(): the whole loop is about 15 lines.

Tool	Does	Why
search_knowledge_base(query)	retrieve() in the knowledge index	Policies, FAQ, product help
search_legal_base(query)	retrieve() in the legal index	Rights, terms, privacy
calculate_date(start_date, days)	date + days	LLMs get dates wrong
escalate_to_human(summary)	returns a hand-off message	Customer asks for a person

6. Python ideas used (and why)

Idea	What it is	Why here	Where
Plain functions + docstrings	One job per function; Args/Returns explained	Easy to read and test step by step	every file
Module constants	UPPER_CASE values at the top of a file	One place to change settings	config.py
load_dotenv(), os.getenv()	Read the .env file / a variable	Keep the API key out of the code	config.py 11
Default arguments	region="US", k=TOP_K	Callers only pass what differs	retriever.py 10

Idea	What it is	Why here	Where
Returning a tuple	return answer, documents	Give the answer and its sources together	rag_chain.py, agent.py
Closure	Inner function uses outer variables	Tools see stores, region, sources without the LLM choosing them	agent.py 17
@tool decorator	Wraps a function	Name + docstring + type hints become the tool description sent to OpenAI	agent.py 33-58
Type hints (query: str)	Declare argument types	@tool needs them to build the tool's argument schema	agent.py
lambda	A one-line function	Region filter passed to FAISS	retriever.py 26
Module-level dict	vector_stores = {}	Load the indexes once, reuse for every question	ui.py 18
dict.fromkeys(...)	Keys keep order, no duplicates	List each source document once	ui.py 56
Path.rglob("*.md")	Find files in all sub-folders	Load every document	loader.py
if __name__ == "__main__"	Runs only when the file is started	python -m app.ingest / app.ui	ingest.py, ui.py
Subclass (tests)	class X(Embeddings)	Fake models: tests run free and offline	tests/

In plain words: **No @dataclass or @staticmethod here - on purpose.** A dataclass is a short way to write a class that only holds data; LangChain's **Document** (text + metadata) already does that job. A static method is a function placed inside a class; with no classes, plain functions are simpler. Use them when you need a container for your own data (dataclass) or a helper that belongs to a class (static method).

7. LangChain pieces used

Piece	What it does	File
ChatOpenAI, OpenAIEmbeddings	Talk to the OpenAI chat and embedding APIs	models.py
Document	Text + metadata (title, region …)	loader.py, chunker.py
MarkdownHeaderTextSplitter, RecursiveCharacterTextSplitter	Split by heading, then by size	chunker.py
FAISS	Vector index: from_documents, similarity_search, save/load	vector_store.py, retriever.py
ChatPromptTemplate	Prompt with {placeholders}	prompts.py
prompt | llm | StrOutputParser()	A chain: each step's output feeds the next	rag_chain.py
bind_tools, tool_calls, ToolMessage	Offer tools · read requests · return results	agent.py

8. AI words in one line

Word	Meaning
Embedding / vector	A list of numbers for a text; similar meaning = close numbers.
Chunk	A small piece of a document, the unit we search.
Context	The retrieved chunks, joined into one text and given to the LLM.
Tool calling	The LLM answers "run search_legal_base(query=…)" instead of text.
Agent loop	LLM → tools → results → LLM … until it answers without a tool.
Temperature 0	Little randomness: same question, same answer.

9. Call sequence: who calls whom

How to read the call trees: read top to bottom, in the order the code runs (the number). An indented box is called *inside* the box above it. The right column shows what each call **returns** to its caller, so the final response travels back up the tree to the first box.

our function | data / search | OpenAI call | loop | returns early

9.1 Ingestion (build the indexes)

CALL (order · function)	WHERE	RETURNS (goes back up to the caller)
python -m app.ingest (or UI button → build_indexes())	`ingest.py 58`	
1 build_all_indexes()	`ingest.py 41`	→ *{"knowledge": FAISS, "legal": FAISS}*
2 get_embedding_model()	`models.py 23`	→ *OpenAIEmbeddings*
3 build_index(KNOWLEDGE_DIR, "knowledge", model)	`ingest.py 14`	→ *FAISS store*
4 load_documents(folder, base)	`loader.py 36`	→ *list[Document] (9 files)*
5 read_markdown(path) - once per file	`loader.py 18`	→ *(metadata, body)*
6 chunk_documents(documents)	`chunker.py 16`	→ *list[Document] (chunks)*
7 clean_text(text) - once per document	`preprocess.py 10`	→ *tidy text*
8 split_text() → split_documents()	`chunker.py 34-37`	→ *sections → small pieces*
9 create_vector_store(chunks, model)	`vector_store.py 14`	→ *FAISS store*
10 FAISS.from_documents → OpenAI embeddings	`vector_store.py 25`	→ *1 vector per chunk*
11 save_vector_store(store, "knowledge")	`vector_store.py 28`	→ *files in storage/knowledge*
12 build_index(LEGAL_DIR, "legal", model)	`ingest.py 14`	→ *same calls again (5 files)*

Figure 6 - Ingestion call tree. build_index() runs twice: knowledge base, then legal base.

- **Down the tree**: build_all_indexes → build_index → load → chunk → embed → save.
- **Back up**: each build_index returns a FAISS store; build_all_indexes returns both in a dict. Nothing goes to the customer - the result is the index files in storage/.

9.2 Mode 1 - Simple RAG chain

CALL (order · function)	WHERE	RETURNS (goes back up to the caller)
Customer clicks Ask → ask_btn.click	ui.py 116	
1 answer_question(question, region, mode)	ui.py 60	→ *(answer, sources, steps) → UI*
2 remove_card_numbers(question)	preprocess.py 25	→ *safe question*
3 get_llm()	models.py 12	→ *ChatOpenAI*
4 get_vector_stores()	ui.py 21	→ *{knowledge, legal}*
5 load_vector_store() x2 - first time only	ui.py 29-31	→ *FAISS from storage/*
6 create_qa_chain(llm, create_qa_prompt())	rag_chain.py 11	→ *prompt | llm | parser*
7 generate_answer(question, stores, qa_chain, region)	rag_chain.py 26	→ *(answer, documents)*
8 retrieve(question, knowledge store, region)	retriever.py 10	→ *4 chunks*
9 similarity_search → embeds the question	retriever.py 23	→ *nearest chunks*
10 retrieve(question, legal store, region)	retriever.py 10	→ *4 chunks*
11 format_context(documents)	retriever.py 31	→ *CONTEXT text "[1] …"*
12 qa_chain.invoke({region, context, question})	rag_chain.py 46	→ *answer text*
13 prompt → ChatOpenAI (gpt-4.1) → parser	OpenAI API	→ *AIMessage → str*
14 format_sources(documents)	ui.py 46	→ *source titles*

Figure 7 - Simple chain call tree. Fixed order: search both bases, then one OpenAI call.

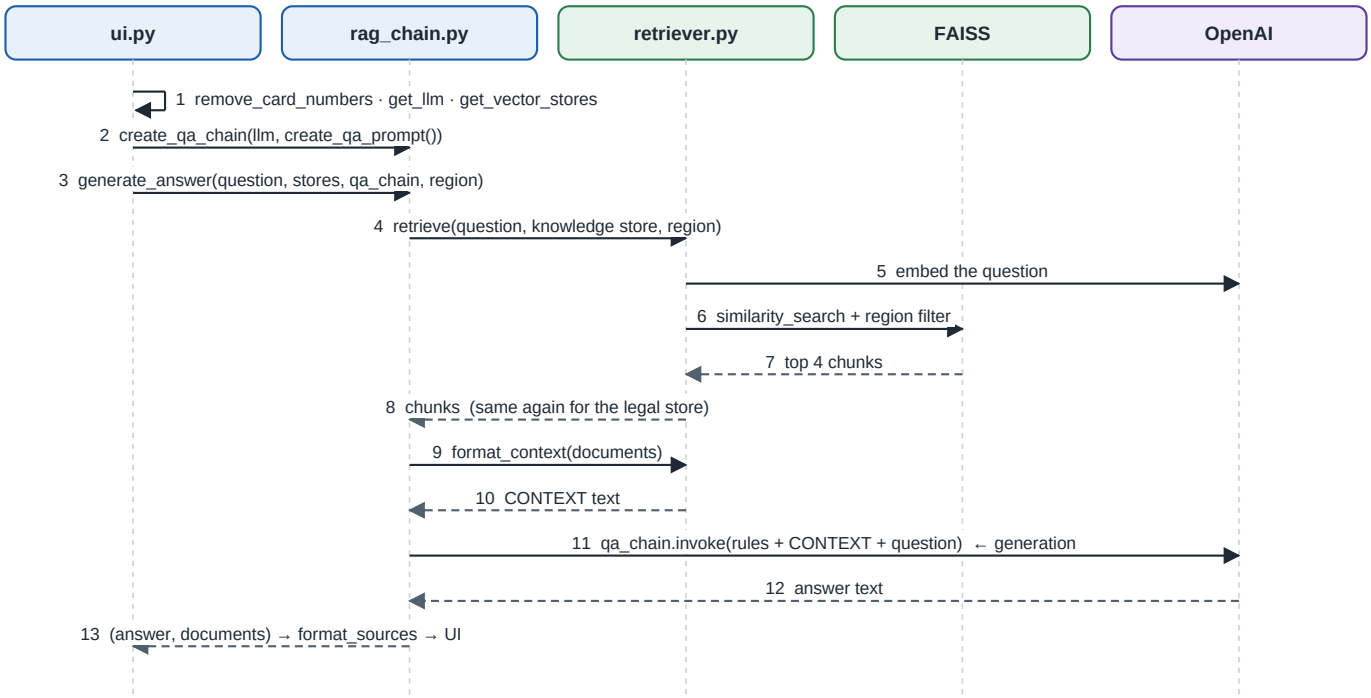


Figure 8 - The same flow as a sequence between files. Dashed = returned value.

- **Final response path**: OpenAI answer → qa_chain.invoke → generate_answer returns (answer, documents) → answer_question returns (answer, sources, steps) → Gradio shows them in the three text boxes.
- **Always 1 LLM call** (plus 2 question embeddings). The code, not the LLM, decides to search.

9.3 Mode 2 - Agent (tool calling)



Figure 9 - Agent call tree. The amber box repeats: the LLM decides each turn whether to call a tool or answer.

- **Turn 1**: invoke → the LLM returns tool_calls → tool.invoke runs search_knowledge_base → retrieve → the CONTEXT is added as a ToolMessage.
- **Turn 2**: invoke again, now with the CONTEXT → no tool_calls → **return the answer** (line 91) → run_agent returns (answer, steps) → answer_question → Gradio.
- Figure 4 (section 5) shows the same flow as a sequence between files.

	Mode 1: Simple RAG chain	Mode 2: Agent
Entry	answer_question() → generate_answer()	answer_question() → run_agent()
Who decides to search	Our code (always both bases)	The LLM (asks for a tool)
How context reaches the LLM	Inside the prompt: "Context: … Question: …"	As a ToolMessage in the message list
OpenAI chat calls	1	2 or more (max 5)
Returns to the UI	(answer, sources, "no tools")	(answer, sources, list of tool calls)

10. Run it

```
pip install -r requirements.txt
cp .env.example .env          # OPENAI_API_KEY=sk-...
python -m app.ingest          # build the indexes (step A)
python -m app.ui              # http://localhost:7860  - pick region and mode, ask
pytest                        # 9 offline tests with fake models
```